

Data Structures and Algorithms Notes

Contents

1	Arrays	2
1.1	Kadane's Algorithm	2
1.2	Bubble Sort	2
1.3	Selection Sort	3
1.4	Insertion Sort	3
1.5	Merge Sort	4
1.6	Binary Search	5
2	Linked Lists	5
2.1	Doubly Linked Lists	7
2.2	Circular Linked Lists	7
3	Stacks	7
4	Queues	7
4.1	Double Ended Queue (Deque)	9
4.2	Circular Queue	9
5	Binary Trees	10
5.1	Traversals	10
5.1.1	DFS based traversals	11
5.1.2	BFS based traversals	11
6	Binary Search Trees (BSTs)	14
6.1	Building a BST	15
6.2	Searching a BST	15
6.3	Deleting a Node in BST	15
7	Graphs	16
7.1	Storing Graphs	16
7.2	Graph Traversals	18
7.2.1	Breadth First Search (BFS)	18
7.2.2	Depth First Search (DFS)	20
7.3	Cycle Detection	21
7.3.1	Cycle Detection in an Undirected Graph	21
7.3.2	Cycle Detection in a Directed Graph	23
7.3.3	Topological Sorting	23
7.3.4	Shortest Paths	23

7.3.5	Minimum Spanning Trees	23
7.3.6	Topological Sorting	23

1 Arrays

1.1 Kadane's Algorithm

```
// maximum subarray

vector<int> nums;

int res = 0;
int currSum = 0;

for(int i = 0; i< nums.size(); i++){
    currSum += nums[i];
    if(currSum < 0){
        currSum = 0;
    }
    res = max(res, currSum);
}
// return res;
```

1.2 Bubble Sort

```
vector<int> nums;

for(int i = 0; i< nums.size()-1; i++){
    bool swapped = false;
    for(int j = 0; j < nums.size()-i-1; j++){
        if(nums[j] > nums[j + 1]){
            swap(nums[j], nums[j+1]);
            swapped = true;
        }
    }
    if(!swapped){
        break;
    }
}
// O(n^2)
// swaps the adjacent entries and push the largest at the last in each loop
```

1.3 Selection Sort

```
vector<int> nums;

for(int i = 0; i < nums.size()-1; i++){
    int minIdx = i;
    for(int j = i+1; j < nums.size(); j++){
        if(nums[j] < nums[minIdx]){
            minIdx = j;
        }
    }
    if(minIdx != i){
        swap(nums[i], nums[minIdx]);
    }
}
// O(n^2)
// find the minimum in the array and places it at its right place in each loop
```

1.4 Insertion Sort

```
vector<int> nums;

for(int i = 1; i < nums.size(); i++){
    int key = nums[i];
    int j = i-1;

    while(j >= 0 && nums[j] > key){
        nums[j+1] = nums[j];
        j--;
    }
    nums[j+1] = key;
}
// O(n^2) for general case, and O(n) for nearly sorted arrays
// places a number in its correct place in the sorted part of the array
```

1.5 Merge Sort

```
void merger(vector<int>& nums, int left, int mid, int right){
    int n1 = mid-left+1;
    int n2 = right-mid;

    vector<int> leftArr(n1), rightArr(n2);

    for(int i = 0; i<n1; i++){
        leftArr[i] = nums[left+i];
    }
    for(int i = 0; i<n2; i++){
        rightArr[i] = nums[mid+1+i];
    }

    int i = 0, j = 0, k = left;
    while(i < n1 && j < n2){
        if(leftArr[i] >= rightArr[j]){
            nums[k] = leftArr[i];
            i++;
        }else{
            nums[k] = rightArr[j];
            j++;
        }
        k++;
    }

    while(i < n1){
        nums[k++] = leftArr[i++];
    }
    while(j < n2){
        nums[k++] = rightArr[j++];
    }
}

void mergeSortHelper(vector<int>& nums, int left, int right){
    if(left > right) return;

    int mid = left + (right - left)/2;
    mergeSortHelper(nums, left, mid);
    mergeSortHelper(nums, mid+1, right);

    merge(nums, left, mid, right);
}

void mergeSort(vector<int>& nums){
    if(nums.size() <= 1) return;
    mergeSortHelper(nums, 0, nums.size()-1);
}

// O(nlogn)
// divide and conquer approach, divides the parts recursively and then merge them
// in a sorted fashion
```

1.6 Binary Search

```
int binarySearch(vector<int>& nums, int key){
    int n = nums.size();
    int l = 0;
    int r = n-1;

    while(l <= r){
        int mid = l + (r-l)/2;
        if(nums[mid] == key) return mid;
        else if(nums[mid] > key){
            r = mid-1;
        }else{
            l = mid+1;
        }
    }
    return -1;
}
// O(log n)
```

2 Linked Lists

Arrays have exceptionally good memory structure if we want to add values in the end or the start. But it's very complicated to add values in the middle or a specific index. Linked Lists solve exactly this problem.

```

# defining node
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# add to the front

// add to the front
void addFront(Node*& head, int value){
    Node* newNode = createNode(value);
    newNode->next = head;
    head = newNode;
}

// insert at end
void insertBack(Node*& head, int value){
    if(!head){
        Node* newNode = createNode(value);
        head = newNode;
        return;
    }
    Node* temp = head;
    while(temp->next){
        temp = temp->next;
    }
    Node* newNode = createNode(value);
    temp->next = newNode;
}

// print the list
void printList(Node* head){
    if(!head){
        cout << "LL is empty" << endl;
        return;
    }
    Node* temp = head;
    while(temp){
        cout << temp ->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

// delete the list
void deleteList(Node* head){
    while(head){
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

int main(){
    Node* head = NULL;
    addFront(head, 10);
    addFront(head, 20);
    addFront(head, 30);
    insertBack(head, 40);
    printList(head);
    deleteList(head);
    return -1;
}

```

2.1 Doubly Linked Lists

```
struct Node{
    int data;
    Node* next;
    Node* prev;

    //constructor
    Node(int value){
        data = value;
        next = prev = NULL;
    }
}
```

2.2 Circular Linked Lists

Everything is the same as above except that we now store a last value in addition to the head node.

3 Stacks

Last In, First Out (LIFO) or First In, Last Out (FIFO).

```
class Stack{
    int arr[100];
    int top;
    int maxSize;

    Stack(){ // constructor
        top = -1;
        maxSize = 100;
    }

    void push(int value){
        if(top >= 100){
            cout<< "Stack OverFlow" << endl;
            return;
        }
        arr[++top] = value;
    }

    void pop(){
        if (top < 1){
            cout << "Stack is Empty" << endl;
        }
    }

};
```

4 Queues

Queues feature first in first out (FIFO) format. Can be implemented using arrays and LLs. It has the following functionalities:

- push() – from the rear end

- pop() – from the front
- front() – gives the top value

All operations take $O(1)$ time.

```
// implementation of a queue using a vector
class Queue{
    vector<int> v;
public:
    void push(int val){
        v.push_back(v);
    }

    void pop(){
        if(v.size() != 0){
            v.erase(v.begin());
        }
    }

    int front(){
        return v[0];
    }

    bool empty(){
        return v.size() == 0;
    }
};
```

```
// queue using a linked list
class Queue{
private:
    list<int> ll;
public:
    void push(int val){
        ll.push(val);
    }

    void pop(){
        if(!ll.empty()){
            ll.pop_front()
        }
    }

    int front(){
        if(!ll.empty()){
            return ll.front();
        }
    }

    bool empty(){
        return ll.empty();
    }
};
```


4.1 Double Ended Queue (Deque)

push → back, front

pop → front, back

front → back

4.2 Circular Queue

Circular Queues have a fixed size. We use an array to create a circular queue. Front and rear pointer are given. Updation will be using modulo to revert back to the start in case of index overflowing.

```
// implementation using an array

class CircularQueue{
    int* arr;
    int currSize, cap;
    int r, f;
public:
    CircularQueue(int size){
        cap = size;
        arr = new int[cap];
        currSize = 0;
        f = 0;
        r = -1;
    }

    void push(int data){
        if(currSize == cap){
            cout << "capacity is full, can't add" << endl;
            return;
        }
        r = (r+1)%cap;
        arr[r] = data;
        currSize++;
    }

    void pop(){
        if(currSize == 0){
            cout<<"there is no element in the queue"<< endl;
            return;
        }
        f = (f+1)%cap;
        currSize--;
    }

    int front(){
        if(currSize == 0){
            cout << "there is no element in the queue"<< endl;
            return -1;
        }
        return arr[f];
    }
};
```

5 Binary Trees

Hierarchical Data Structure. Main Node $\rightarrow$ root. There are parent nodes and child nodes, with each parent having only at most 2 children nodes, namely, left child and the right child. Leaf nodes are the ones that have no children.

```
// building a binary tree
```

```
class Node{
    int data;
    Node* left;
    Node* right;

    Node(int val){
        data = val;
        left = right = NULL;
    }
};
```

```
// making the tree from the preorder sequence (root, left subtree, right subtree)
```

```
static int idx = -1;
Node* buildTree(vector<int> preorder){
    idx++;
    if(preorder[idx] == -1) return NULL;

    Node* root = new Node(preorder[idx]);
    root->left = buildTree(preorder);
    root->right = buildTree(preorder);

    return root;
}
// TC: O(n)
```

In the given preorder sequence, all the leaf nodes are represented by '-1'. Only the root (main node) is required to access the entire tree.

5.1 Traversals

- 3 recursion based (**DFS**)
 - preorder (root, left, right)
 - inorder (left, root, right)
 - postorder (left, right, root)
- 1 iterative (**BFS**)
 - level order traversal

5.1.1 DFS based traversals

```
// preorder traversal -  $O(n)$ 
void preorder(Node* root){
    if(!root) return;

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

```
// inorder traversal -  $O(n)$ 
void inorder(Node* root){
    if(!root) return;

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

```
// postorder traversal -  $O(n)$ 
void postorder(Node* root){
    if(!root) return;

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

5.1.2 BFS based traversals

```
// using a queue
void levelOrder(Node* root){
    queue<Node*> q;
    q.push(root);
    while(!q.empty()){
        Node* curr = q.front();
        q.pop();
        if(curr){
            cout << curr->data << " ";
        }

        if(curr->left){
            q.push(curr->left);
        }
        if(curr->right){
            q.push(curr->right);
        }
    }
}
// TC:  $O(n)$ 
```

```

// printing them in the levels - O(n)
void levelOrder(Node* root){
    queue<Node*> q;
    q.push(root);
    q.push(NULL); // for the next line
    while(!q.empty()){
        Node* curr = q.front();
        q.pop();
        if(curr == NULL){
            if(!q.empty()){
                cout << endl;
                q.push(NULL); // for the next line
                continue;
            }else{
                break;
            }
        }
        cout << curr->data << " ";
        if(curr->left){
            q.push(curr->left);
        }
        if(curr->right){
            q.push(curr->right);
        }
    }
}

```

Height of a BT

```

int height(Node* root){
    if(!root) return -1;

    leftHt = height(root->left);
    rightHt = height(root->right);

    return max(leftHt, rightHt) + 1; // +1 is for the current level
}
// O(n)

```

Counting Nodes

```

int countNode(Node* root){
    if(!root) return 0;

    leftCount = countNode(root->left);
    rightCount = countNode(root->right);

    return leftCount + rightCount + 1;
}
// O(n)

```

Identical Trees

```
bool identicalTrees(Node* p, Node* q){
    if(!p || !q) return p == q; // if any of them is null, check if both are null

    bool leftCheck = identicalTrees(p->left, q->left);
    bool rightCheck = identicalTrees(p->right, q->right);

    return leftCheck && rightCheck && p->data == q->data;
}
// O(n)
```

Subtree of another tree

```
// find subroot in the main tree and check for identical - O(n)
bool isSubtree(Node* root, Node* subroot){
    if(!root || !subroot) return root == subroot;

    if(root->data == subroot->data && identicalTrees(root, subroot)) return true;

    return isSubtree(root->left, subroot) || isSubtree(root->right, subroot);
}
```

Morris Inorder Traversal (Inorder Predecessor) Inorder Predecessor is the rightmost node in the left subtree. The idea in this traversal is to not use a recursion stack but **iterative approach**. In recursive ways, we can reach to the parent nodes via backtracking, but if we know the inorder predecessor, we can simply thread ip->right as the parent node we want to be on. The code to find IP is as follows:

```
// finding inorder predecessor (IP)
Node* findIP(Node* root){
    Node* ip = root->left;
    while(ip->right) ip = ip->right;
    return ip;
}
```

```

// morris inorder traversal
void morrisINO(Node* root){
    Node* curr = root;

    while(curr){
        if(curr->left == NULL){
            cout<< curr->data << " ";
            curr = curr->right;
        }else{
            //find IP
            Node* IP = curr->left;
            while(curr->right && curr->right != curr){ // the second condition
                is to ensure that we don't get stuck in a loop to find the IP
                IP = IP->right;
            }
            if(IP->right == NULL){
                IP->right = curr; // creating the thread
                curr = curr->left;
            }else{
                IP->right = NULL;
                cout << curr->data << " "; // deleting the thread
                curr = curr->right;
            }
        }
    }
}

```

6 Binary Search Trees (BSTs)

BSTs are made such that all the entries in the left subtree are smaller than the parent node and all the values in the right subtree are greater than the parent node. The preorder traversal of a BST is always sorted. Searching in BSTs happens in $O(\text{height})$ or $O(\log(n))$.

6.1 Building a BST

```
// defining Node
class Node{
    int data;
    Node* left;
    Node* right;

    Node(int val){
        data = val;
        left = right = NULL;
    }
};

Node* insert(Node* root, int val){
    if(!root){
        return new Node(val);
    }
    if(val < root->data){
        root->left = insert(root->left, val);
    }else{
        root->right = insert(root->right, val);
    }
    return root;
}

Node* buildBST(vector<int> arr){
    Node* root = NULL;
    for(int val : arr){
        root = insert(root, val);
    }
    return root;
}
```

6.2 Searching a BST

```
bool search(Node* root, int key){
    if(!root) return false;
    if(root->data == key) return true;
    if(root->data > key){
        return search(root->left, key);
    }else{
        return search(root->right, key);
    }
}
// O(height), O(log n) for a balanced tree
```

6.3 Deleting a Node in BST

There are three cases for the node:

- 0 child – chumma delete
- 1 child – the child node becomes the parent node
- 2 child – find the inorder successor, replace the root with IS and then delete the IS

Inorder Successor – Left most entry in the right subtree.

Inorder Predecessor – Right most entry in the left subtree.

```
Node* getIS(Node* root){
    while(root != NULL && root->left != NULL){
        root = root->left;
    }
    return root;
}

Node* deleteNode(Node* root, int key){
    if(!root) return NULL;

    // finding the node with key as the value
    if(root->data > key){
        return deleteNode(root->left, key);
    }else if(root->data < key){
        return deleteNode(root->right, key);
    }else{
        // root->data == key
        if(root->left == NULL){
            Node* temp = root->right;
            delete root;
            return temp;
        }else if(root->right == NULL){
            Node* temp = root->left;
            delete root;
            return temp;
        }else{
            Node* IS = getIS(root->right);
            root->data = IS->data;
            root->right = delNode(root->right, IS->data);
        }
    }
}
```

7 Graphs

7.1 Storing Graphs

There are mainly two ways of storing graphs, i.e.,

- **Adjacency Matrix**
- **Adjacency List**

Adjacency Matrix is a Vertex $\times$ Vertex matrix having 1 and 0s and entries. **Adjacency List** lists all the vertices each vertex has edges with.


```

// graph using adjacency matrix
void addEdge(vector<vector<int>>& mat, int i, int j){
    mat[i][j] = 1;
    mat[j][i] = 1; //undirected graph
}

void displayGraph(vector<vector<int>> mat){
    int V = mat.size();
    for(int i = 0; i<V; i++){
        for(int j = 0; j<V; j++){
            cout << mat[i][j] << " ";
        }
        cout << endl;
    }
}

int main(){
    int V = 4;
    vector<vector<int>> mat(V, vector<int>(V, 0));

    addEdge(mat, 0, 1);
    addEdge(mat, 1, 2);
    addEdge(mat, 3, 4);
    addEdge(mat, 3, 1);

    displayGraph(mat);
    return -1;
}

```

```

// graph using an adjacency list
void addEdge(vector<vector<int>>& adj, int i, int j){
    adj[i].push_back(j);
    adj[j].push_back(i); // undirected graph
}

void displayAdjList(vector<vector<int>>& adj){
    for(int i = 0; i< adj.size(); i++){
        cout << i << ":"; // printing the vertex
        for(int j = 0; j<adj[i].size(); j++){
            cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}

int main(){
    int V = 4;
    vector<vector<int>> adj(V, vector<int>(V, 0));

    addEdge(adj, 0, 1);
    addEdge(adj, 1, 2);
    addEdge(adj, 4, 3);
    addEdge(adj, 2, 4);

    displayAdjList(adj);
    return -1;
}

```

7.2 Graph Traversals

7.2.1 Breadth First Search (BFS)

```
// bfs approach of graph traversal
vector<int> bfs(vector<vector<int>>& adjMat){
    int V = adjMat.size();
    vector<int> res;
    int s = 0; //source node
    queue<int> q;
    q.push(0);

    vector<bool> visited(V, false);
    visited[s] = true;

    while(!q.empty()){
        int curr = q.front();
        q.pop();
        res.push_back(curr);

        for(int i : adjMat[curr]){
            if(!visited[i]){
                visited[i] = true;
                q.push(i);
            }
        }
    }
    return res;
}
```

```

// bfs for disconnected graphs
void bfsHelper(vector<vector<int>>& adjMat, vector<int>& visited, vector<int>& res
, int i){
    queue<int> q;
    q.push(i);
    visited[i] = true;

    while(!q.empty()){
        int curr = q.front();
        q.pop();
        res.push_back(curr);
        for(int k : adjMat[curr]){
            if(!visited[k]){
                visited[k] = true;
                q.push(k);
            }
        }
    }
}

vector<int> bfs(vector<vector<int>>& adjMat){
    int V = adjMat.size();
    vector<int> res;
    vector<bool> visited(V, false);

    for(int i = 0; i<V; i++){
        if(!visited[i]){
            bfsHelper(adjMat, visited, res, i);
        }
    }
    return res;
}

//O(V + E) : TC, SC, O(V)

```

7.2.2 Depth First Search (DFS)

```
// handles disconnected graphs
void dfsHelper(vector<vector<int>>& adjMat, vector<int>& res, vector<int>& visited
, int i){

    visited[i] = true;
    for(int k : adjMat[i]){
        if(!visited[k]){
            dfsHelper(adjMat, res, visited, k);
        }
    }
}

vector<int> dfs(vector<vector<int>>& adjMat){
    int V = adjMat.size();
    vector<int> res;
    vector<bool> visited(V, false);

    for(int i = 0; i<V; i++){
        if(!visited[i]){
            dfsHelper(adjMat, res, visited, i);
        }
    }
    return res;
}

// O(V + E) : TC, SC: O(V + E)
```

7.3 Cycle Detection

7.3.1 Cycle Detection in an Undirected Graph

```
// on gfg
// cycle detection in an undirected graph using bfs
class Solution {
private:
    bool helper(vector<vector<int>>& adj, int src, vector<bool>& visited){

        visited[src] = true;
        queue<pair<int, int>> q;
        q.push({src, -1});

        while(!q.empty()){
            int curr = q.front().first;
            int parent = q.front().second;
            q.pop();

            for(int adjacent : adj[curr]){
                if(!visited[adjacent]){
                    q.push({adjacent, curr});
                    visited[adjacent] = true;

                }else if(parent != adjacent){
                    return true;
                }
            }
        }
        return false;
    }

public:
    bool isCycle(int V, vector<vector<int>>& edges) {

        vector<vector<int>> adj(V);
        for(auto edge : edges){
            int u = edge[0];
            int v = edge[1];

            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<bool> visited(V, false);

        for(int i = 0; i < V; i++){
            if(!visited[i]){
                if(helper(adj, i, visited)) return true;
            }
        }

        return false;
    }
};
```

```

// on gfg
// cycle detection in an undirected graph using dfs
class Solution {

private:
    bool helper(vector<vector<int>>& adj, int src, int parent, vector<bool>&
visited){
        visited[src] = true;
        for(int adjacent : adj[src]){
            if(!visited[adjacent]){
                if(helper(adj, adjacent, src, visited)) return true;
            }else if(parent != adjacent){
                return true;
            }
        }
        return false;
    }

public:
    bool isCycle(int V, vector<vector<int>>& edges) {

        vector<vector<int>> adj(V);
        for(auto edge : edges){
            int u = edge[0];
            int v = edge[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<bool> visited(V, false);
        for(int i = 0; i<V; i++){
            if(!visited[i]){
                if(helper(adj, i, -1, visited)) return true;
            }
        }
        return false;
    }
};

```

7.3.2 Cycle Detection in a Directed Graph

```
// dfs approach from gfg
class Solution {
private:
    bool dfsCheck(vector<vector<int>>& adj, int node, vector<bool>& visited,
        vector<bool>& pathVisited){

        visited[node] = true;
        pathVisited[node] = true;

        for(int adjacent : adj[node]){
            if(!visited[adjacent]){
                if(dfsCheck(adj, adjacent, visited, pathVisited)) return true;
            }else if(pathVisited[adjacent]){ //we have visited and also pathVis = true
                return true;
            }
        }

        pathVisited[node] = false;
        return false;
    }

public:
    bool isCyclic(int V, vector<vector<int>> &edges) {

        vector<vector<int>> adj(V);
        for(auto edge : edges){
            int u = edge[0];
            int v = edge[1];
            adj[u].push_back(v);
        }

        vector<bool> visited(V, false);
        vector<bool> pathVisited(V, false);

        for(int i = 0; i<V; i++){
            if(!visited[i]){
                if(dfsCheck(adj, i, visited, pathVisited)) return true;
            }
        }

        return false;
    }
};
```

There is another method. Basically you try to find the topological sorting of the graph using Kahn's Algorithm. If you are able to do it, then it's not cyclic, otherwise it is.

7.3.3 Topological Sorting

7.3.4 Shortest Paths

7.3.5 Minimum Spanning Trees

7.3.6 Topological Sorting